

Lecture Title: OpenMP Directives and Memory Model

Subject: High Performance Computing (HPC)

Target Audience: Undergraduate/Graduate students in Computer Science

Lecture Duration: 60 Minutes

Tools Required: C Compiler with OpenMP support (e.g., GCC), Text Editor (VSCode/Gedit)

✦ Learning Objectives

By the end of this lecture, students will be able to:

1. Understand the syntax and semantics of key OpenMP directives.
 2. Comprehend OpenMP's memory model and hierarchy.
 3. Use `#pragma omp parallel` and related directives.
 4. Analyze how threads share or partition memory in OpenMP.
 5. Write simple OpenMP programs to demonstrate memory sharing.
-

📘 1. Introduction to OpenMP

- **OpenMP (Open Multi-Processing)** is an API that supports **multi-platform shared memory multiprocessing programming** in C, C++, and Fortran.
 - It uses **compiler directives**, **runtime library routines**, and **environment variables** to manage parallelism.
-

📖 2. OpenMP Directives: Syntax and Semantics

OpenMP directives start with:

`#pragma omp directive [clauses]`

✅ Common Directives:

Directive	Purpose
<code>#pragma omp parallel</code>	Starts a parallel region

#pragma omp for	Splits loop iterations across threads
#pragma omp sections	Defines multiple code sections for threads
#pragma omp single	Restricts a block to a single thread
#pragma omp barrier	Synchronizes all threads at a point
#pragma omp critical	Ensures mutual exclusion (no race)

Example 1: Basic Parallel Region

```
#include <stdio.h>

#include <omp.h>

int main() {
    #pragma omp parallel
    {
        int id = omp_get_thread_num();
        printf("Hello from thread %d\n", id);
    }
    return 0;
}
```

Output (on 4 threads):

```
Hello from thread 0
Hello from thread 2
Hello from thread 1
Hello from thread 3
```

3. OpenMP Memory Model & Hierarchy

Key Concepts:

- **Shared Memory:** Variables accessible by all threads.
- **Private Memory:** Each thread has its own copy.
- **Thread Stack:** Where private variables reside.

Memory Scopes:

Scope	Description
Shared	All threads can access and modify
Private	Thread-specific variables
Firstprivate	Private + initialized from master thread
Lastprivate	Final iteration value copied back to master

Example 2: Private vs Shared Variables

```
#include <stdio.h>
```

```
#include <omp.h>
```

```
int main() {
```

```
    int shared_var = 100;
```

```
    #pragma omp parallel private(shared_var)
```

```
    {
```

```
        shared_var = omp_get_thread_num();
```

```
        printf("Thread %d has shared_var = %d\n", omp_get_thread_num(), shared_var);
```

```
    }
```

```
    printf("Main thread sees shared_var = %d\n", shared_var);
```

```
    return 0;
```

```
}
```

Observation:

- Even though `shared_var` is declared globally, it becomes **private inside the parallel region** due to the directive.
-

4. #pragma omp parallel and Other Directives

#pragma omp parallel

- Creates a **team of threads** to execute a block of code.

#pragma omp for

- Used inside a parallel region to **distribute loop iterations**.

```
#include <stdio.h>
```

```
#include <omp.h>
```

```
int main() {  
    int i;  
    #pragma omp parallel for  
    for(i = 0; i < 8; i++) {  
        printf("Thread %d executes iteration %d\n", omp_get_thread_num(), i);  
    }  
    return 0;  
}
```

5. Thread-Level Memory Sharing and Partitioning

Shared Memory

- All threads **can read and write** shared variables.
- Needs care: Can lead to **data races**.

Private Memory

- Threads maintain **individual copies** of variables.
-

⚠ Data Race Example

```
#include <stdio.h>

#include <omp.h>

int main() {
    int counter = 0;

    #pragma omp parallel for
    for(int i = 0; i < 1000; i++) {
        counter++; // Unsafe: data race
    }

    printf("Counter = %d\n", counter); // Likely wrong!
    return 0;
}
```

✅ Fix using #pragma omp atomic:

```
#pragma omp atomic
counter++;
```

🔧 6. Code Snippets and Hands-On Demos

👛 Firstprivate Example

```
#include <stdio.h>

#include <omp.h>

int main() {
    int x = 10;
```

```
#pragma omp parallel firstprivate(x)
{
    x += omp_get_thread_num();
    printf("Thread %d x = %d\n", omp_get_thread_num(), x);
}

printf("Original x = %d\n", x); // Will remain 10
return 0;
}
```

Lastprivate Example

```
#include <stdio.h>
#include <omp.h>

int main() {
    int i, result;

    #pragma omp parallel for lastprivate(result)
    for(i = 0; i < 5; i++) {
        result = i * 2;
    }

    printf("Result after loop = %d\n", result); // From last iteration
    return 0;
}
```

7. Summary and Key Takeaways

- OpenMP provides **easy-to-use directives** for parallelism.
 - **Shared vs private** memory management is crucial.
 - Understand and avoid **race conditions** using synchronization tools.
 - Use **firstprivate/lastprivate/atomic/critical** wisely.
-

Assignment

Write a C program using OpenMP:

- Create 4 threads.
- Use `#pragma omp parallel for` to sum numbers 1 to 100.
- Avoid race conditions.